

硬件交互规则

基本原则：验证到底层

永远不要假设任何硬件相关的API是非阻塞的、安全的或行为符合预期的。在使用任何HAL函数或硬件接口之前：

1. 追踪调用链到寄存器级别
2. 检查调用链中是否有任何函数可能阻塞
3. 确认项目中实际使用的HAL库版本
4. 检查项目是否自定义了任何底层实现（标准HAL行为可能已被修改）

阻塞分析

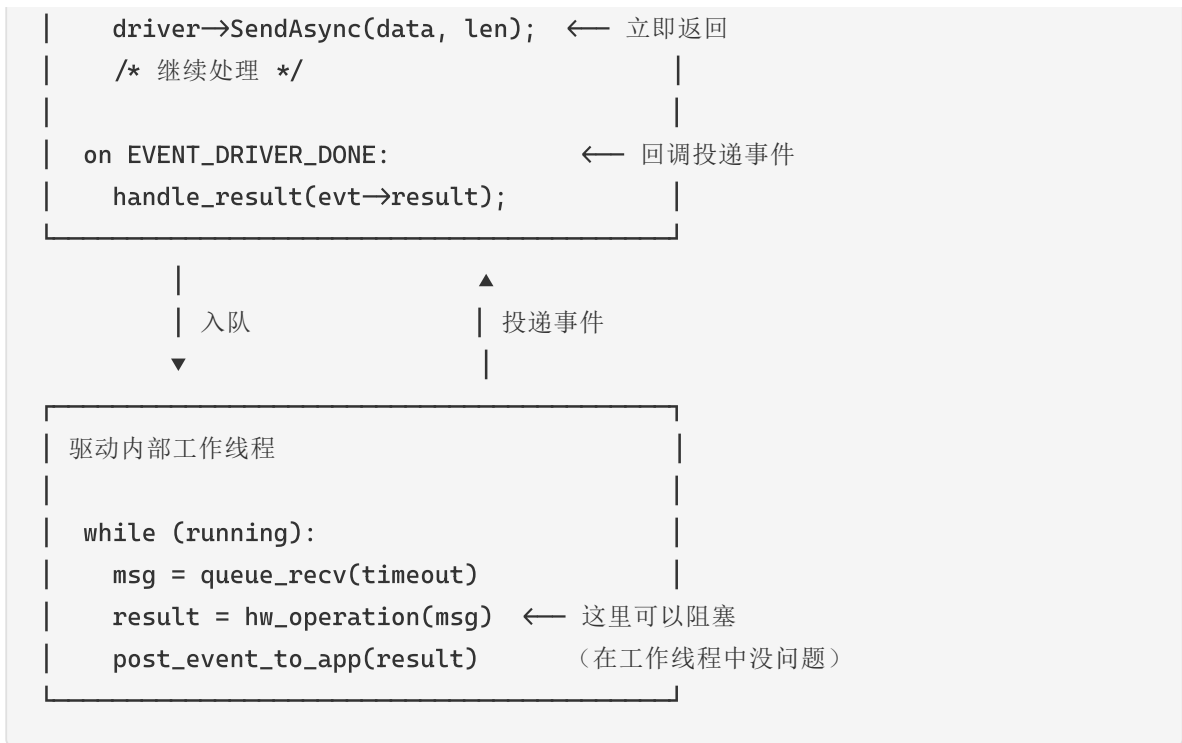
什么算阻塞

- 忙等循环（轮询状态寄存器）
- `HAL_Delay()` 或任何延时函数
- 在轮询模式下等待DMA完成
- 等待外设就绪标志
- Flash写入/擦除操作
- I2C/SPI轮询模式下的事务

验证流程

在调用任何硬件函数之前：

1. 打开函数实现
2. 检查每一行是否有：
 - └─ 等待硬件的 `while/for` 循环
 - └─ 调用延时函数
 - └─ 调用其他函数（递归检查它们）
 - └─ 超时参数（意味着可能阻塞）



周期性轮询

如果驱动需要周期性硬件轮询：

- 轮询定时器/循环在驱动的工作线程内部运行
- 驱动仅对外暴露异步API和回调
- 应用层绝不直接调用轮询函数
- 轮询间隔可配置，但默认值应定义为宏

平台相关注意事项

HAL库版本

不同的HAL版本对相同API可能有不同的行为。始终检查：

1. 项目使用的HAL版本
2. 是否有HAL源文件被本地修改
3. 是否有weak回调函数被重写
4. DMA通道/流是否按预期配置

FatFS注意事项

FatFS的不同版本差异很大。在使用任何FatFS API之前：

1. 检查项目中的FatFS版本 (`ffconf.h`)
2. 确认启用了哪些可选功能 (`FF_USE_LFN` 、 `FF_FS_REENTRANT` 等)
3. 检查磁盘I/O层实现 (`diskio.c`)
4. FatFS操作可能阻塞——仅在工作线程中使用
5. 在多线程调用前检查 `FF_FS_REENTRANT` 是否已启用

中断优先级

当驱动使用中断时：

1. 文档标明所需的中断优先级
2. 确保不会与RTOS管理的中断发生优先级反转
3. ISR回调必须极简——仅设置标志/投递到队列
4. 绝不在ISR中分配内存或调用复杂函数
5. 使用项目现有的中断优先级方案

ISR到应用层的信号架构

关键设计规则——源于历史教训：

当GPIO中断（或任何硬件ISR）需要通知应用层（如QPC活动对象）时，**始终使用高优先级工作线程模式**，而非直接ISR回调。

错误方案（最初设计，后来不得不返工）：

```
ISR → callback(status) → Q_NEW_FROM_ISR / QF_PUBLISH_FROM_ISR
```

直接ISR回调的问题： - 强制回调使用ISR安全的API（FromISR变体） - 发布的事件进入队列尾部——无法将紧急信号（如掉电）优先于已排队的文件写入 - 将驱动设计与ISR约束耦合 -

`QF_publish()` 没有LIFO变体； `QACTIVE_POST_LIFO()` 仅在任务级可用，无法从ISR调用

正确方案（高优先级工作线程模式）：

```
ISR → osSemaphoreRelease（极简，ISR安全）
↓
高优先级线程（osPriorityHigh）唤醒 → 读取硬件 → 回调
↓
回调在任务上下文中运行 → 可使用任何RTOS/QPC API
→ QACTIVE_POST_LIFO 用于紧急信号（队列头部）
→ QACTIVE_POST 用于普通信号（队列尾部）
```

优势： - ISR极简（仅一次信号量释放） - 回调在任务上下文运行——可用完整API - 应用层可选择LIFO或FIFO投递方式 - 驱动与QPC/RTOS细节解耦 - 高优先级线程抢占低优先级AO线程，确保信号在AO 恢复处理之前完成投递

设计任何新的中断驱动驱动时： 1. 默认使用高优先级工作线程模式 2. ISR仅释放信号量或设置线程标志 3. 让应用层注册回调 4. 让应用层决定信号传递机制（publish、post、post-LIFO） 5. 驱动不得包含QPC头文件或了解QActive对象

CMSIS-RTOS2 线程/信号量创建规则

关键——历史教训导致设备启动崩溃：

osThreadNew() 属性约束

FreeRTOS的CMSIS-RTOS2封装对 `osThreadAttr_t` 的内存字段 有严格的配对规则：

cb_mem	stack_mem	结果
NULL	NULL	动态分配（正确）
已提供	已提供	静态分配（正确）
NULL	已提供	非法——返回NULL → 崩溃
已提供	NULL	非法——返回NULL → 崩溃

要么同时提供 `cb_mem` + `stack_mem`，要么两者都不提供。

在调用 `osThreadNew()` 之前：

1. **搜索项目中** 现有的 `osThreadNew` 调用并严格匹配其属性模式
2. 如果项目使用 `cb_mem = NULL, cb_size = 0` 且没有 `stack_mem`（全动态），就遵循该模式
3. 绝不将静态栈与动态TCB混用，反之亦然

```
/* 错误 - stack_mem 没有配对 cb_mem → 崩溃 */
const osThreadAttr_t attr = {
    .name = "my_thread",
    .stack_mem = my_stack,          /* 已提供 */
    .stack_size = sizeof(my_stack),
    .priority = osPriorityHigh,
    /* cb_mem 未设置 → NULL → 非法组合 */
};
```

```
/* 正确 - 全动态（匹配项目约定） */
const osThreadAttr_t attr = {
    .name = "my_thread",
    .cb_mem = NULL,
    .cb_size = 0,
    .stack_size = 512,
    .priority = osPriorityHigh,
};
```

osSemaphoreNew() — 调度器启动前可安全调用

`osSemaphoreNew()` 可以在 `osKernelStart()` 之前调用。信号量会被创建，但在调度器运行之前没有任务可以等待它。根据CMSIS-RTOS2规范，`osSemaphoreRelease()` 是ISR安全的。

DMA + 环形缓冲区非阻塞接收架构

这是嵌入式串口驱动最常用的高性能非阻塞接收方案。后续会有配套视频详细讲解，B站搜索「兆鸣嵌入式」观看。

架构

```
硬件UART → DMA自动搬运 → 环形缓冲区 → 应用层读取
          |
          | 中断通知（Idle Line / Half/Full Transfer）
          |
          | 释放信号量 → 唤醒等待的任务
```

关键组件

1. **环形缓冲区 (Ring Buffer)**：DMA写入端，应用读取端，读写索引分离无需加锁
2. **DMA配置为循环模式 (Circular)**：硬件自动搬运，不需要CPU参与
3. **利用UART Idle Line检测**（`HAL_UARTEx_ReceiveToIdle_DMA`）实现不定长接收
4. **ISR中仅做信号量释放**，不处理数据
5. **读写索引分离**：ISR更新写索引，应用任务更新读索引

实现要点

```

/* DMA循环模式接收初始化 */
static uint8_t s_dma_buf[DMA_RX_BUF_SIZE];
static ring_buffer_t s_rx_ring;

static void uart_rx_start(uart_drv_t *self)
{
    HAL_UARTEx_ReceiveToIdle_DMA(
        self->huart, s_dma_buf, DMA_RX_BUF_SIZE);
    /* 关闭Half Transfer中断（如不需要） */
    __HAL_DMA_DISABLE_IT(
        self->huart->hdmarx, DMA_IT_HT);
}

/* Idle Line回调 - 在ISR上下文 */
void HAL_UARTEx_RxEventCallback(
    UART_HandleTypeDef *huart, uint16_t Size)
{
    /* 更新环形缓冲区写索引 */
    ring_buffer_update_write_index(&s_rx_ring, Size);
    /* 仅释放信号量，不处理数据 */
    osSemaphoreRelease(s_rx_sem);
}

```

D-Cache注意事项

使用DMA时如果MCU有D-Cache（如Cortex-M7），接收前必须 无效化（Invalidate）对应缓冲区的Cache行，否则CPU读到的 可能是Cache中的旧数据而非DMA写入的新数据。

```

/* 读取DMA数据前，无效化Cache */
SCB_InvalidateDCache_by_Addr(
    (uint32_t *)s_dma_buf, DMA_RX_BUF_SIZE);

```

或者将DMA缓冲区放在非Cache区域（通过MPU配置或链接器脚本）。

错误恢复

UART错误（帧错误、溢出等）发生时：

1. 在错误回调中 abort 当前DMA传输
2. 重新启动DMA接收
3. 通过回调通知上层应用

```
void HAL_UART_ErrorCallback(  
    UART_HandleTypeDef *huart)  
{  
    HAL_UART_DMAStop(huart);  
    ring_buffer_reset(&s_rx_ring);  
    uart_rx_start(s_uart_drv); /* 重新启动 */  
    /* 通知上层 */  
    if (s_uart_drv->error_cb != NULL) {  
        s_uart_drv->error_cb(s_uart_drv,  
                               huart->ErrorCode);  
    }  
}
```

寄存器级验证检查清单

编写涉及硬件寄存器的代码时：

- ☐ 寄存器地址和位域定义正确
- ☐ 需要时使用正确的读-改-写序列
- ☐ 所有硬件寄存器指针使用volatile限定符
- ☐ 外设访问宽度正确（8/16/32位）
- ☐ 寄存器访问之间的必要延时（如需要）
- ☐ 访问前已启用外设时钟
- ☐ 引脚复用/GPIO复用功能已配置

获取更多

- **B站搜索「兆鸣嵌入式」** 观看完整教程（详细讲解+代码实战）
- 公众号搜索「兆鸣嵌入式」回复「交流」加技术交流群
- 抖音搜索「兆鸣嵌入式」看短视频精华版